

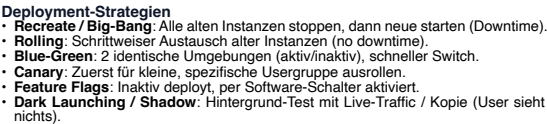
- On-demand Self

- **Web-Fehler:** Bugs, Deployment, Skalierung, Ausfälle, Monitoring, Security.
- **Container-Issues (2025):** Kultur (47%), Training/Security (36%), CI/CD/Monitoring (35%), Komplexität (34%).

Phase	Aktivitäten	Tools
Plan	Requirements, Roadmaps, Architektur, Security-Pläne	Jira, Trello
Code	Code schreiben, lokale Tests, Peer Reviews, Gated Merges	GitLab, GitHub
Build	Kompilieren, Container-Images, Artefakte pushen	GitLab CI, Docker
Test	Unit-/Integrationstests, E2E, Performance, Security	JUnit, SonarQube
Release	Verifizierte Artefakte pushen, Tagging, Release Notes	Artifactory, Cosign
Deploy	Manifeste anwenden, Strategien, Post-Validierung	Argo CD, Terraform
Operate	Infra betreuen, Skalierung, Patches, Backups	Vault, Velero
Monitor	SLIs/SLOs, Dashboards, Alerts, Logs	Prometheus, Datadog



- **CI (Cont. Integration):** Autom. Build/Test bei Push (Plan, Code, Build, Test).
- **CDel (Cont. Delivery):** Autom. bis Staging, Prod-Deploy manuell.
- **CDep (Cont. Deployment):** Vollautom. Deploy in Prod ohne Mensch.



- Infrastruktur abstrahieren
- Compliance & Security erzwingen
- Observability bieten

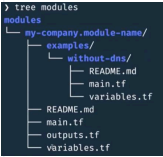
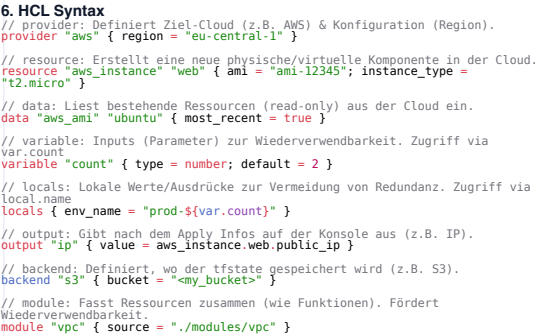
1. Konzepte

- ### 3. Imperativ vs. Deklarativ

Merkmal	Imperativ	Deklarativ
Fokus	Wie? (Schritt-für-Schritt)	Was? (Zielzustand)
Zustand	Skripte fallen oft (nicht idempotent)	Nur nötiges ändert sich (idempotent)
Tools	Bash, AWS CLI	OpenTofu, Terraform

- **Language:** HCL (HashiCorp Config. Language).

- ## 5. Terraform Workflow



1. Konzepte & Modelle

- ## 2. Ansible Core

- ### 3. Task, Play & Playbook



```
- name: Setup Webserver
```

- **Handlers:** Spezielle Tasks (nur 1x am Ende), laufen nur wenn durch notify getriggert
- **Loops (loop):** Führt einen Task iterativ über eine Liste aus (siehe oben).
- **Roles:** Strukturieren Playbooks in wiederverwendbare Ordner (tasks, vars, templates)
- **Tags:** Markieren Tasks, um gezielt nur diese auszuführen (CLI: `-tags config`)
- **Ansible-Template-Tagging:** Dynamische Dateien & Variablen-Einsatz (z.B. `{% for tag %}`).
- **Collections:** Größere Pakete, die mehrere Roles, Modules und Plugins bündeln (Download via `ansible-galaxy`).

Jinja2-Template (.j2):

5. Ansible CLI Commands

- **Ping:** ansible all -i inventory.ini -m ping
- **Ad-Hoc:** ansible webbservers -i inventory.ini -m apt -a "name=curl state=present" --become
- **Playbook:** ansible-playbook -i inventory.ini site.yml
- **Syntax:** ansible-playbook -i inventory.ini site.yml --syntax-check
- **Dry-Run:** ansible-playbook -i inventory.ini site.yml --check

1. Core Konzepte

- ## 2. Pipeline Beispiel (.gitlab-ci.yml)

```
test_job:
  stage: test
  image: node:18
```

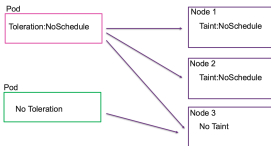


- **API Server:** Zentraler Eingangspunkt Alle

-
- The diagram illustrates the architecture of a Kubernetes cluster. It is divided into two main sections: the **Control Plane** and the **Worker Nodes**.
- Control Plane:** This section is enclosed in a dashed blue box. It contains five master nodes, each represented by a blue hexagon with a white cloud icon. These nodes are interconnected with solid and dashed lines. A central master node is connected to the **Cloud provider API** (a blue cloud icon) and the **API server** (a blue hexagon with a white cloud icon). The **API server** is connected to the **Cloud controller manager** (a blue hexagon with a white cloud icon), which in turn connects to the **Controller manager** (a blue hexagon with a white cloud icon). The **Controller manager** is connected to the **Scheduler** (a blue hexagon with a white cloud icon). The **Scheduler** is connected to the **Kube proxy** (a blue hexagon with a white cloud icon), which is connected to the **Kubernetes** (a blue hexagon with a white cloud icon). The **Kubernetes** component is connected to the **etcd** (a blue hexagon with a white cloud icon), which is connected to the **etcd** (a blue hexagon with a white cloud icon). The **etcd** is connected to the **etcd** (a blue hexagon with a white cloud icon).
- Worker Nodes:** This section contains three nodes, each represented by a blue rectangle with a white cloud icon. Each node contains two worker nodes, each represented by a blue hexagon with a white cloud icon. The worker nodes are connected to the **Kubernetes** component in the Control Plane.
- Legend:** A vertical list of components with their corresponding icons:
- API server (blue hexagon with white cloud icon)
 - Cloud controller manager (blue hexagon with white cloud icon)
 - Controller manager (blue hexagon with white cloud icon)
 - etcd (blue hexagon with white cloud icon)
 - Kubernetes (blue hexagon with white cloud icon)
 - Kube proxy (blue hexagon with white cloud icon)
 - Scheduler (blue hexagon with white cloud icon)
 - Control plane (blue dashed box)
 - Node (blue rectangle with white cloud icon)

- #### 4. Deployment Update Strategies
- **Recreate:** Alte Pods löschen, dann neue erstellen. (Downtime! Nötig, falls Versionen nicht parallel laufen dürfen).
 - **RollingUpdate** (Default): Aktualisiert Pods nacheinander für Zero-Downtime.
 - `maxUnavailable`: Max. Anzahl an Pods, die während dem Update offline sein dürfen.

- ## 5. Taints & Tolerations
- **Taints:** Werden auf **Nodes** gesetzt und „stoßen“ unerwünschte Pods ab.
 - **Tolerations:** Werden auf **Pods** gesetzt und erlauben (erzwingen aber nicht) das Scheduling auf Nodes mit passendem Taint.



- ## 6. Probes (Health Checks)
- **readinessProbe:** Ist der Pod bereit für Traffic? (Pod ip wird erst bei Erfolg an Services weitergeleitet).
 - **livenessProbe:** Läuft die App noch oder ist sie eingefroren? (Startet Pod bei Fehler neu).
 - **startupProbe:** Für langsame Legacy-Apps. Pausiert liveness-Checks während der langsamen Initialisierung.

7. Deployment YAML & InitContainer

```

apiVersion: apps/v1
kind: Deployment
metadata: { name: my-app }
spec:
  replicas: 2
  selector: { matchLabels: { app: my-app } }
  template:
    metadata: { labels: { app: my-app } }
    spec:
      initContainers: # Laufen sequenziell VOR den normalen Containern
                     (müssen erfolgreich beenden)
      - name: init-setup
        image: busybox
        command: ['sh', '-c', 'echo "Setup done"']
      containers:
      - name: web
        image: nginx:alpine

```

8. K8S PACKAGE & CONFIG MANAGEMENT

- 1. Helm (Package Manager mit Templating)**
 - Charts:** So nennt Helm seine Pakete. Enthalten alle YAML-Manifeste (Templates) für eine App.
 - Values (values.yaml):** Variablen, die beim Deployment dynamisch in die Templates eingesetzt werden.
- 2. Releases:** Jede laufende Instanz eines Charts auf dem Cluster.

Wichtige Chart .yaml Felder:

- apiVersion:** v2 (für Helm 3), v1 (für ältere Versionen / Backwards-Compatibility).
- appVersion:** Version der deployten App (nur Info, hat keinen Einfluss auf die Chart-Version).
- kubeVersion:** Unterstützte K8s-Versionen (z.B. >= 1.12.1 < 1.14.0). Installation bricht ab, wenn inkompatibel.

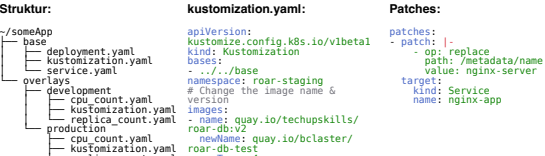
Template & Values Beispiel:

```
# values.yaml
replicaCount: 3

# deployment.yaml (Template)
spec:
  replicas: {{ .Values.replicaCount }}
```

2. Kustomize (Config Management ohne Templating)

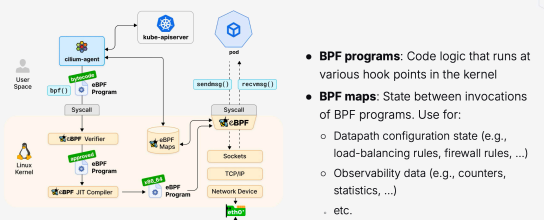
- Patches:** Prinzip: Nutzt reale, valide YAML-Dateien. Man definiert eine „Basis-Konfiguration“ (**Base**) und erstellt für verschiedene Umgebungen (Dev, Prod) spezifische **Overlays**.
- Overlays:** Verändern die Base-Konfiguration umgebungsspezifisch.
- Patches:** Gezielte YAML-Snippets, die bestehende Base-Ressourcen überschreiben/ergänzen.
- Transformers:** Passen Ressourcen automatisiert an (z.B. **allen** Ressourcen ein bestimmtes Label anfügen).
- Components:** Wiederverwendbare, einkapselbare Konfigurationsblöcke (flexibler als Standard-Bases).



Components:	Overlays mit Components:
ext-storage:	my-settings:
dev:	test:
prod:	

9. CILIU

Linux kernel + eBPF



Automatically Allowed Connections

- Wenn keine NetworkPolicies definiert sind, ist Traffic standardmäßig erlaubt (Default Deny greift erst, sobald eine Policy einen Pod selektiert).
- Ingress/Egress:** Ohne matchende Policy wird nichts blockiert.
- Worker Nodes:** Ingress von K8s Worker Nodes immer erlaubt (verhindert Blockieren von Kubelet Liveness/Readiness Probes).
- Host-Network:** Connectivity von Host-Network Pods ist immer erlaubt (Host-Network = trusted).

Policy Layers

- L3 (Network Endpoints):** to/fromEndpoints, to/fromCIDR (Set), to/FQDN.
 - L4:** to/fromPort.
 - L7:** DNS, HTTP, gRPC, Kafka, Memcached, Cassandra.
- Policy Entities & Identities (toEntities/fromEntities)**
- world:** Alle Endpoints außerhalb des Clusters.
 - cluster:** Alle gemanagten Pod-Endpoints.
 - host:** Der lokale K8s Worker Node.
 - remote-node:** Externe K8s Worker Nodes.
 - kube-apiserver:** Remote Nodes mit laufendem API-Server.

Default Deny Policy

Hebt das „Automatic Allow“ auf durch leere Regeln für Ingress & Egress (blockiert jeglichen Traffic). App-Teams müssen explizite „Allow“-Ausnahmen deklarieren.

Achtung: Blockiert auch Namespace-internen Traffic und Zugriff auf CoreDNS / K8s API!

Deny All:	Allow Within Namespace:
-----------	-------------------------

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: deny-all
spec:
  endpointSelector: {}
  ingress:
  - {}
  egress:
  - {}
```

Feature-Rich Policy:

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: feature-rich-policy
spec:
  endpointSelector:
    matchLabels:
      app: backend # Ziel-Pods (AND)
  ingress:
  - fromEndpoints: # OR: Regel 1
    matchLabels: # Pods mit app=frontend
  AND env=prod
  - fromEndpoint:
    env: prod
    matchNamespaces: # UND im Namespace
  ns=prod
  matchLabels:
    ns: prod
  toPorts:
  - ports:
    - port: "80"
    protocol: TCP
  rules: # L7 HTTP-Filterung
    http:
      method: "GET"/".*"
  egress:
  - toCIDRSet: # OR: Egress zu bestimmten IPs
    cidr: "192.168.1.0/24"
    except:
      - "192.168.1.50/32"
  - toEndpoints: # OR: DNS-Zugriff
    matchLabels:
      k8s:io.kubernetes.pod.namespace: kube-system
      k8s-app: kube-dns
  toPorts:
  - ports:
    - port: "53"
    protocol: UDP
  rules:
    dns:
      matchPattern: "/*.mycompany.com"
```

10. SERVICE MESH

Microservices Pros & Cons

- Pros:**
 - Independent Deployment
 - Flexibility & Scalability
 - Fault Isolation
 - Team Autonomy
- Cons:**
 - Operational Complexity
 - Distributed System Challenges
 - Data Consistency issues
 - Higher Resource Usage
 - Increased Testing Overhead

Service Mesh Core Features

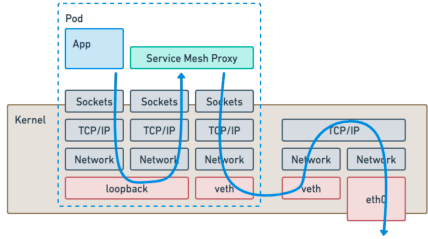
- Traffic Management:** Fine-grained Routing, Traffic Splitting.
 - Resilience:** Fault Tolerance, Automatic Retries & Failback.
 - Security:** Encrypted Communication (mTLS), Authentication, Authorization.
 - Observability:** Metrics, Logs, Tracing.
- Use Cases**
- Canary / Blue-Green:** Stufenweises Traffic-Shifting zwischen App-Versionen.
 - Circuit Breaking:** Verhindert kaskadierende Ausfälle durch schnellen Fail-Fast.
 - Rate Limiting:** Schützt nachgelagerte Services vor Überlastung.
 - Zero-Trust Security:** Verschlüsselt (mTLS) und authentifiziert jeden Aufruf.

Architecture (Data & Control Plane)

- Data Plane:** Leichtgewichtige Proxies (z.B. Envoy) als Sidecars (Kommunikation, Security, Metriken).
- Control Plane:** Verwaltet Konfigurationen & Policies (z.B. Istiod) und verteilt Regeln dynamisch.
- Core Components & Tools:**
 - Istio Ingress Gateway:** Envoy Proxy als Einstiegspunkt für den Traffic in das Service Mesh.
 - Istiod:** Backbone der Control Plane (Konfiguration von Proxies und Richtlinien).
 - Jaeger:** Bietet Tracing-Funktionalität für Traffic innerhalb des Service Mesh.
 - Kiali:** Graphisches Dashboard zur Visualisierung von Struktur, Traffic und Health im Mesh.

Istio Sidecar Downsides

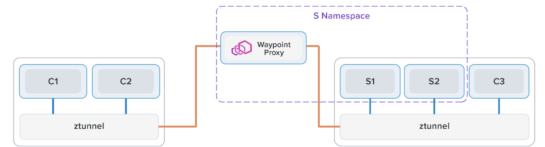
- Invasiveness:** Modifizierte Pod-Spec & umgeleiteter Traffic. Service-Mesh-Updates erfordern einen Pod-Neustart.
- Underutilization:** Hohe Overhead-Ressourcen (CPU/Mem) für jeden Workload-Proxy. Ressourcen müssen für Worst-Case-Szenarien provisioniert werden.



Istio Ambient Mode

- Splitlöst Service-Mesh-Funktionen in zwei Schichten auf (ohne Sidecars):
- ztunnel (Zero-Trust Tunnel):** Läuft als DaemonSet pro Node. Handhabt L4-Transport (mTLS, Verschlüsselung, Routing, Identität, Telemetry, Metriken) für alle Pods auf dem Node.

- Waypoint Proxy:** Optionaler L7-Proxy (Envoy) pro Namespace / Service-Account. Wird nur bei Bedarf für L7-Policies (HTTP-Routing, Auth, Rate Limiting) zwischen-geschaltet.



K8s Gateway API Routing (HTTPRoute)

HTTPRoute Canary (Traffic Splitting):

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-app-route
spec:
  parentRefs:
  - name: my-gateway
  rules:
  - backendRefs:
    - name: frontend
      port: 5000
      weight: 90
    - name: frontend-v2
      port: 5000
      weight: 10
```

HTTPRoute Shadow (Request Mirroring):

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-app-mirror
spec:
  parentRefs:
  - name: my-gateway
  rules:
  - backendRefs:
    - name: frontend
      port: 5000
    - type: RequestMirror
      requestMirror:
        backendRef:
          name: frontend-v2
          port: 5000
```

Waypoint Proxy Deployment

- Ein Waypoint-Proxy wird **pro Namespace** (oder Service-Account) deployt. **Wichtig:** L7-Ressourcen (VirtualService, HTTPRoute) greifen im Ambient Mode erst, wenn ein Waypoint-Proxy im Namespace läuft (andernfalls nur L4 per ztunnel).
- CLI:** istioctl waypoint apply -n my-namespace
- YAML (Gateway API):**

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: waypoint
  namespace: my-namespace
spec:
  gatewayClassName: istio-waypoint
  listeners:
  - name: mesh
    port: 15008
    protocol: HBONE
```

11. OBSERVABILITY

Monitoring vs. Logging

Monitoring (Metrics/trends - Quantitative):

- Utilization (Memory, Disk, CPU, ...)
- Counters (Requests, Error counts, ...)
- Rates (Error Rate, throughput, ...)

Push vs. Pull Monitoring

- Push (Agent-based):**
 - Agent-Software sendet Daten aktiv an Collector.
 - Vorteil:** System entscheidet, wann gesendet wird.
 - Nachteil:** Agent-Installation nicht überall möglich.
- Pull (Agentless):**
 - Collector fragt System ab (HTTP, SNMP, ICMP).
 - Vorteil:** Collector steuert Abfragefrequenz.
 - Nachteil:** Überwachtes System muss erreichbar sein.

Scaling Prometheus

- Federated Prometheus:** Hierarchische Föderation. Übergeordneter Prometheus (z.B. per Region) scrapet selektierte Metriken von untergeordneten Instanzen.
- Thanos:** Sidecar-basierte Föderation. Bietet globale Abfragen, Langzeitspeicherung (Object Storage) und Deduplizierung.
- Grafana Mimir:** Horizontal skalierbar mit Microservices (Ingester, Querier). Unterstützt Multi-Tenancy; nutzt Remote-Write (Push).

PromQL Examples

- Counter (total):** Braucht rate(...[5m]), um Geschwindigkeit pro Sekunde zu berechnen.
- Gauge:** Braucht kein rate(), da es bereits den aktuellen Zustand darstellt.

- Pods per Namespace:** sum by (namespace) (kube_pod_info)
- Containers without CPU limits:**

```
count by (namespace) (
  sum by (namespace, pod, container) (
    kube_pod_container_info{container!=""}
  ) unless sum by (namespace, pod, container) (
    kube_pod_container_resource_limits{resource="cpu"}
  )
)
```

- Pod Restarts by Namespace (5m):** sum by (namespace) changes(kube_pod_status_ready{condition="true"}[5m])
- Pods not ready:** sum by (namespace) (kube_pod_status_ready{condition="false"})
- CPU Overcommit:** sum(kube_pod_container_resource_limits{resource="cpu"}) - sum(kube_node_status_capacity_cpu_cores)
- Memory Overcommit:** sum(kube_pod_container_resource_limits{resource="memory"}) - sum(kube_node_status_capacity_memory_bytes)
- Ready Nodes:** sum(kube_node_status_condition{condition="Ready", status="true"}=1)
- Node Readiness Flapping (15m):** sum(changes(kube_node_status_condition{status="true", condition="Ready"}[15m]) by (node) > 2)
- CPU Idle:**

```
sum(
  rate(container_cpu_usage_seconds_total{container!="POD", container!=""}[30m])
  on (namespace, pod, container) group_left
  avg by (namespace, pod, container) (
    kube_pod_container_resource_requests{resource="cpu"}
  )
) < 1 > 0
```

Memory Idle (in GB):

```
sum(
  container_memory_usage_bytes{container!="POD", container!=""}
  on (namespace, pod, container)
  avg by (namespace, pod, container) (
    kube_pod_container_resource_requests{resource="memory"}
  )
)
```

```
} * 1 > 0
) / (1024 * 1024 * 1024)
```

Logging Architectures

- Node Level (DaemonSet):** Ein Logging-Agent läuft auf jedem Node.
 - + Funktioniert auch bei Container-Absturz.
 - Versand ist gebatcht (nicht zwingend Echtzeit).
- App Level (Direct Push):** Applikation pusht Logs direkt an Backend.
 - + Niedrige Latenz (Echtzeit).
 - Komplexität in App; Logverlust bei Verbindungsabbruch/frühem Crash.
- Sidecar:** Separater Logging-Container im selben Pod liest & leitet Logs weiter.

Grafana Loki & Alloy

- Loki:** „Prometheus für Logs“. Indexiert nur Metadaten/Labels (nicht den Logtext selbst) -> extrem kleiner Index & sehr kosteneffizient. LogQL für Queries.
- Alloy:** Universeller Open-Source Telemetrie-Collector (Grafana Agent Nachfolger). Integriert OTel, Prometheus & Loki; sammelt Metrics, Logs, Traces & Profiles deklarativ in einer Pipeline.

LogQL Queries (Loki)

Log Queries (Output: Log-Zeilen):

```
{app="api"} |= "error"
```

Line-Filter (Text-Inhalt):

- |= : Zeile enthält String.
- != : Zeile enthält String nicht.
- ~ : Zeile matcht Regex.
- !~ : Zeile matcht Regex nicht.

Metric Queries (Output: Time Series):

```
rate({app="log-generator"}) |
= "176" [1m]
```

Label-Filter (Metadaten):

- == (Gleichheit), != (Ungleichheit)
- >, >=, <, <= (Vergleich), and / or (Ketten)

12. GITOPS & CD

Push-based vs. Pull-based GitOps

Push-based (Imperativ):

- + Einfach zu nutzen & bekannt aus klassischem CI/CD.
- + Schnelle Deployment-Geschwindigkeit.
- Erfordert offene Firewall/Admin-Rechte für externes CI/CD.
- Pipeline-Anpassung bei jeder neuen Umgebung nötig.
- Configuration Drift ist schwerer zu kontrollieren.

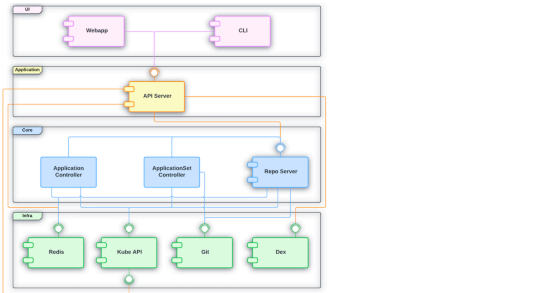
Pull-based (Deklarativ / GitOps):

- + Sicherer: Keine offene Firewall / externe Admin-Rechte nötig.
- + Änderungen werden autom. erkannt/angewendet (Drift-Korrektur).
- + Einfache Skalierung identischer Cluster.
- Benötigt In-Cluster Agent (z.B. Argo CD).
- Moderate Deploy-Geschwindigkeit (Polling-Interval / Sync).

Argo CD vs. Flux CD

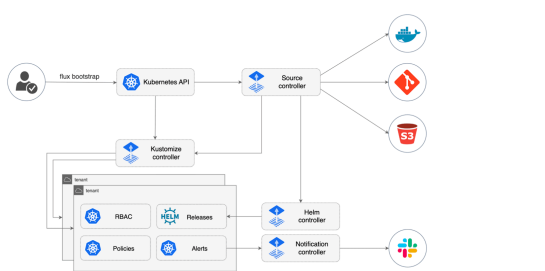
Argo CD:

- Features:** Web UI, Multi-Cluster Management, SSO-Integration, App-of-Apps Pattern.
- Ressourcen:** Application, ApplicationSet, AppProject.



Flux CD:

- Features:** GitOps Toolkit (modulare Controller), native K8s-Integration, CLI-driven.
- Ressourcen:** GitRepository, OCIRepository, Bucket, HelmRepository, Customization, HelmRelease.



Secrets Management in GitOps

- Sealed Secrets:** Asymmetrisch. Verschlüsselung erfolgt lokal via kubeseal (Public Key), Entschlüsselung im Cluster durch Controller (Private Key) in native K8s Secrets.
- Mozilla Ispas:** Symmetrische Datei-Verschlüsselung (nur Values verschlüsselt -> Git Diff losbar). Nutzt KMS (AWS, Vault, etc.) oder GPG/Aes. Entschlüsselung in CI/CD oder durch In-Cluster Operator.
- External Secrets (ESO):** Pull-Modell. Ein Operator fragt externe Secrets-Manager (Vault, AWS Secrets Manager, etc.) ab und synchronisiert Werte in native K8s Secrets.
- Secrets Store CSI Driver:** Mountet Secrets aus externen Providern direkt als Volume (RAM-Mount via CSI-Speichertreiber) in die Pods. Kein Ablegen im etcd notwendig.